

1. HASH FUNCTION – MD5

AIM

To implement the MD5 hash function and generate the hash value for a given message using Python.

ALGORITHM

1. Start the program.
2. Read the input text from the user.
3. Convert the text into bytes.
4. Apply the MD5 hashing algorithm.
5. Generate the hash value.
6. Display the hash value.
7. Stop the program.

PYTHON CODE

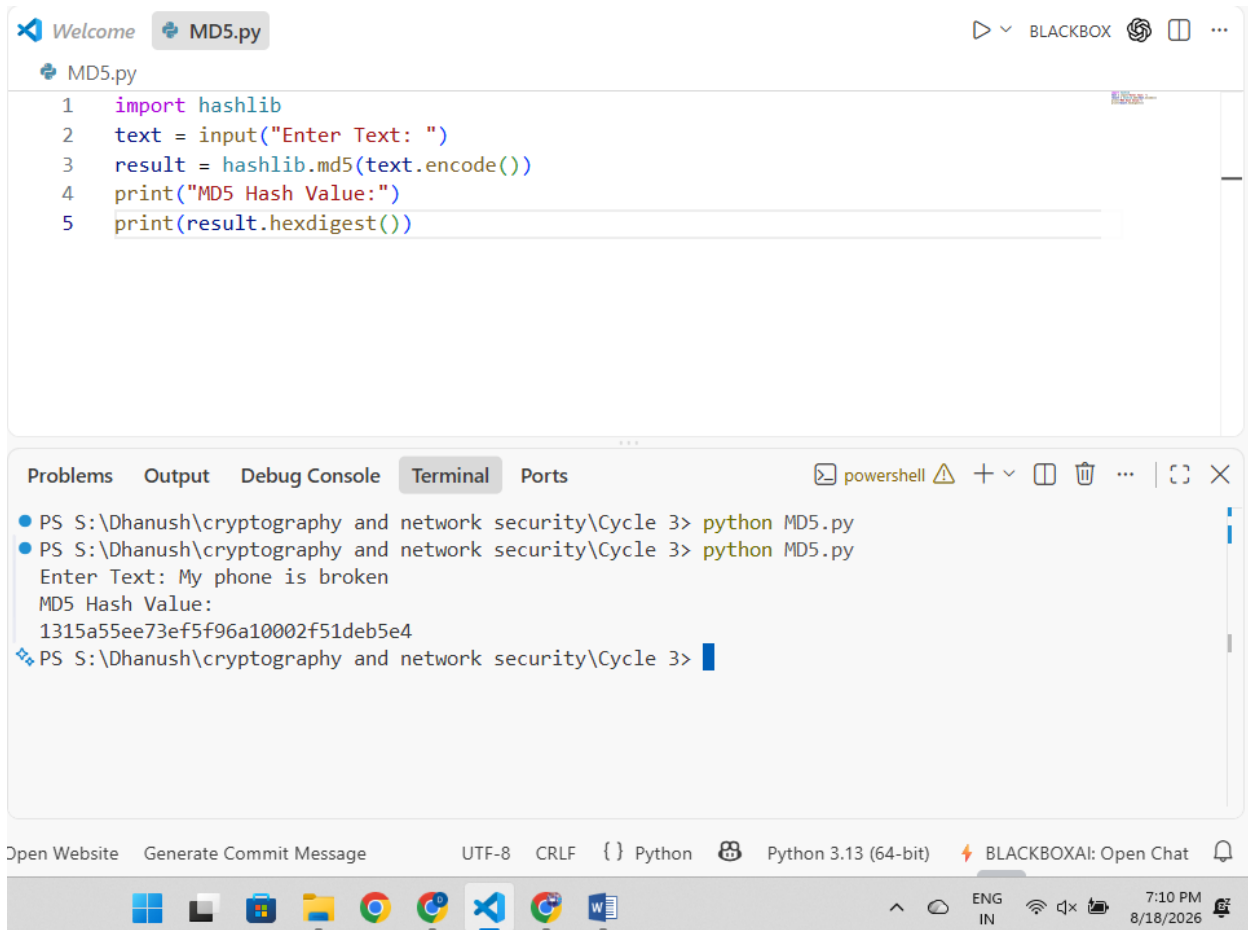
```
import hashlib

text = input("Enter Text: ")

result = hashlib.md5(text.encode())

print("MD5 Hash Value:")
print(result.hexdigest())
```

SAMPLE OUTPUT



The screenshot displays the Visual Studio Code interface. The editor window shows a file named `MD5.py` with the following Python code:

```
1 import hashlib
2 text = input("Enter Text: ")
3 result = hashlib.md5(text.encode())
4 print("MD5 Hash Value:")
5 print(result.hexdigest())
```

Below the editor, the `Terminal` panel is active, showing the execution of the script in a PowerShell environment. The output is as follows:

```
PS S:\Dhanush\cryptography and network security\Cycle 3> python MD5.py
PS S:\Dhanush\cryptography and network security\Cycle 3> python MD5.py
Enter Text: My phone is broken
MD5 Hash Value:
1315a55ee73ef5f96a10002f51deb5e4
PS S:\Dhanush\cryptography and network security\Cycle 3>
```

The status bar at the bottom indicates the file encoding is UTF-8, the line ending is CRLF, and the language is Python. It also shows the Python version as 3.13 (64-bit) and a connection to BLACKBOXAI.

RESULT

Thus, the MD5 hash function was implemented successfully and the hash value was generated.

2. HASH FUNCTION – SHA512

AIM

To implement the SHA512 hash function and generate the hash value for a given message using Python.

ALGORITHM

1. Start the program.
2. Read the input text from the user.
3. Convert the text into bytes.
4. Apply the SHA512 hashing algorithm.
5. Generate the hash value.
6. Display the hash value.
7. Stop the program.

PYTHON CODE

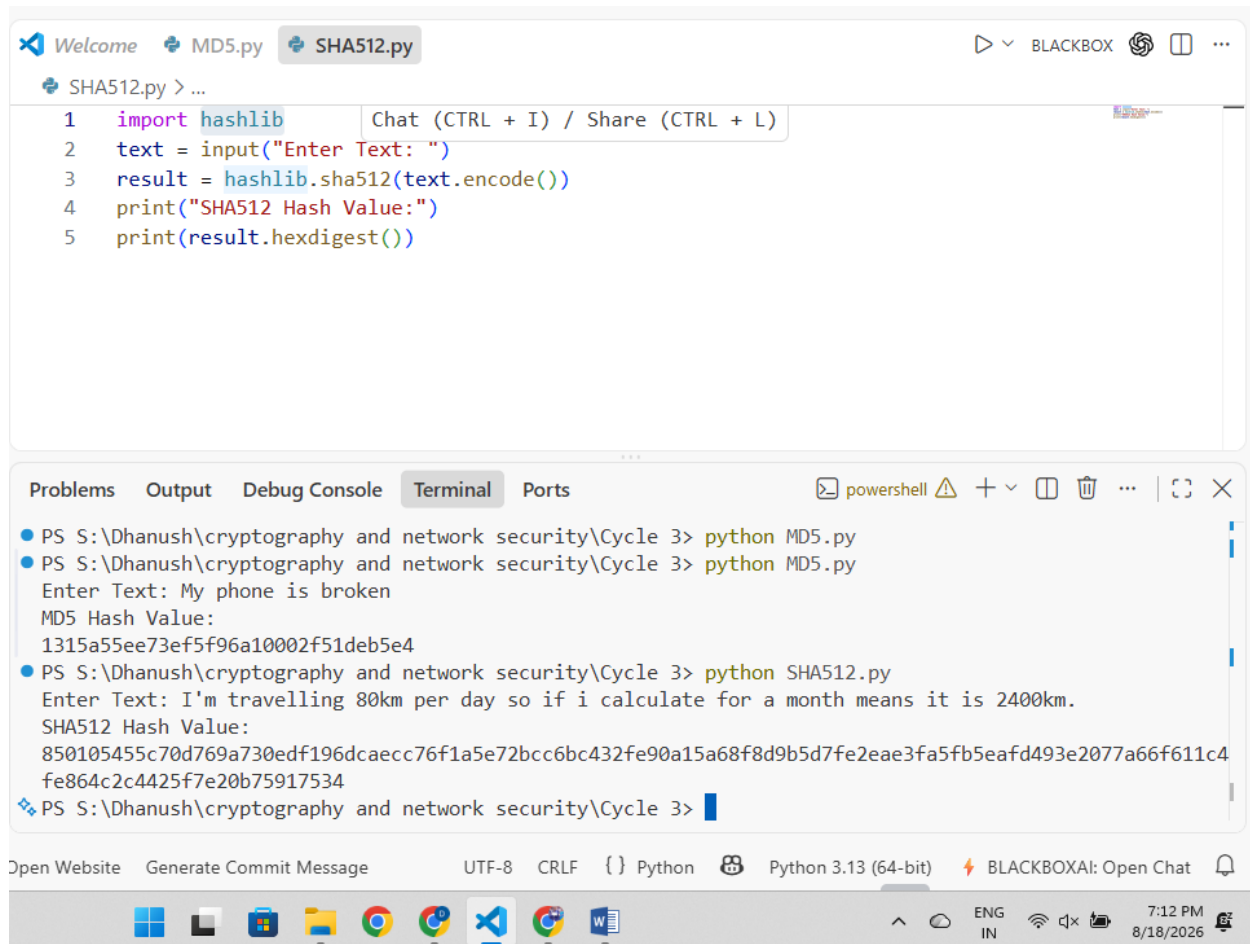
```
import hashlib

text = input("Enter Text: ")

result = hashlib.sha512(text.encode())

print("SHA512 Hash Value:")
print(result.hexdigest())
```

SAMPLE OUTPUT



The screenshot displays the Visual Studio Code interface. The editor window shows the `SHA512.py` script with the following code:

```
1 import hashlib
2 text = input("Enter Text: ")
3 result = hashlib.sha512(text.encode())
4 print("SHA512 Hash Value:")
5 print(result.hexdigest())
```

The terminal window shows the execution of the script. It first runs `python MD5.py`, which prompts for input and outputs the MD5 hash. Then, it runs `python SHA512.py`, which prompts for input and outputs the SHA512 hash.

```
PS S:\Dhanush\cryptography and network security\Cycle 3> python MD5.py
PS S:\Dhanush\cryptography and network security\Cycle 3> python MD5.py
Enter Text: My phone is broken
MD5 Hash Value:
1315a55ee73ef5f96a10002f51deb5e4
PS S:\Dhanush\cryptography and network security\Cycle 3> python SHA512.py
Enter Text: I'm travelling 80km per day so if i calculate for a month means it is 2400km.
SHA512 Hash Value:
850105455c70d769a730edf196dcaecc76f1a5e72bcc6bc432fe90a15a68f8d9b5d7fe2eae3fa5fb5eafd493e2077a66f611c4
fe864c2c4425f7e20b75917534
PS S:\Dhanush\cryptography and network security\Cycle 3>
```

The bottom status bar shows the file encoding as UTF-8, line endings as CRLF, and the Python version as 3.13 (64-bit). The system tray at the bottom indicates the time as 7:12 PM on 8/18/2026.

RESULT

Thus, the SHA512 hash function was implemented successfully and the hash value was generated.

3. HASH FUNCTION – SHA1

AIM

To implement the SHA1 hash function and generate the hash value for a given message using Python.

ALGORITHM

1. Start the program.
2. Read the input text from the user.
3. Convert the text into bytes.
4. Apply the SHA1 hashing algorithm.
5. Generate the hash value.
6. Display the hash value.
7. Stop the program.

PYTHON CODE

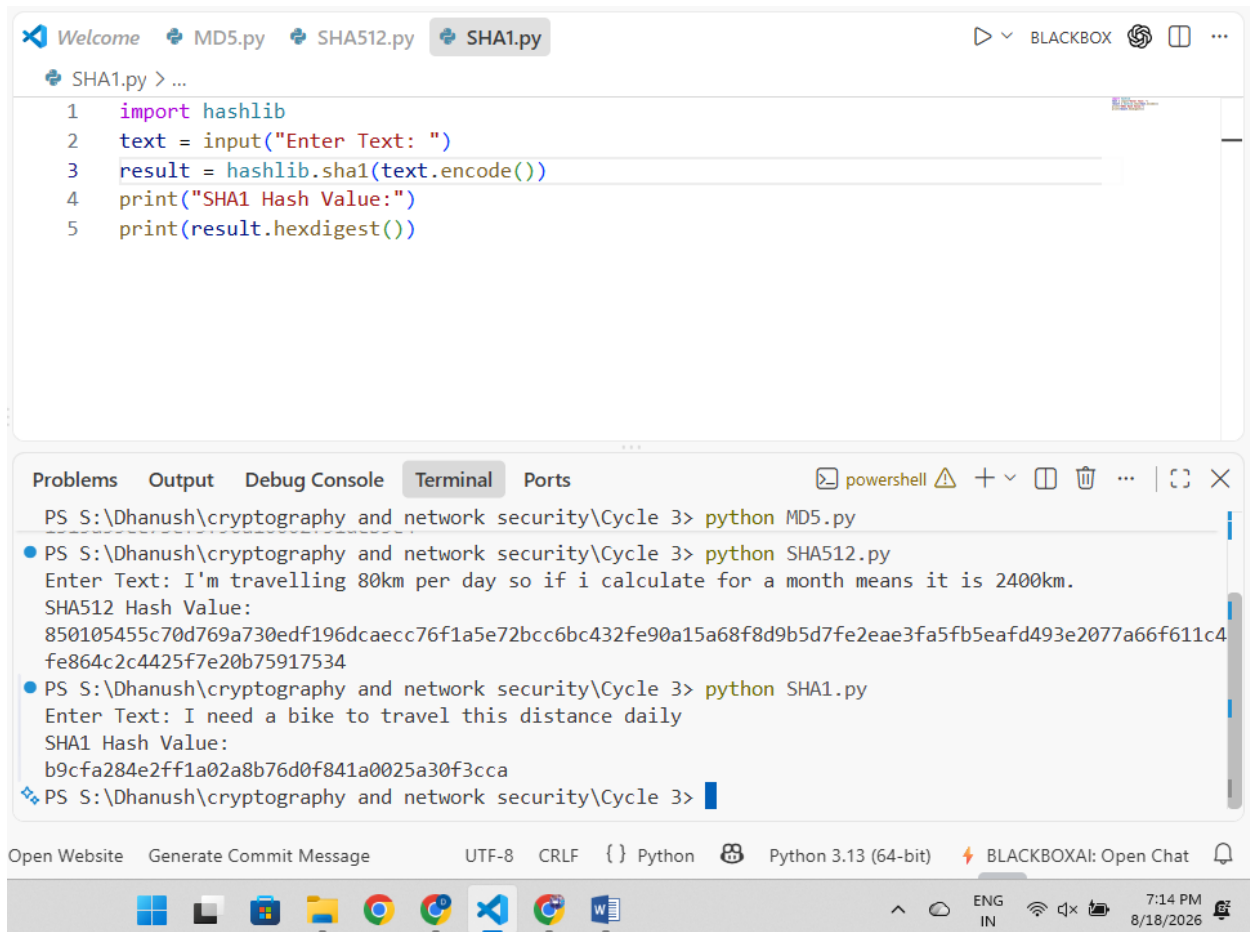
```
import hashlib

text = input("Enter Text: ")

result = hashlib.sha1(text.encode())

print("SHA1 Hash Value:")
print(result.hexdigest())
```

SAMPLE OUTPUT



The screenshot displays the Visual Studio Code interface. The editor window shows a file named `SHA1.py` with the following Python code:

```
1 import hashlib
2 text = input("Enter Text: ")
3 result = hashlib.sha1(text.encode())
4 print("SHA1 Hash Value:")
5 print(result.hexdigest())
```

The terminal window at the bottom shows the execution of the script. It contains the following output:

```
PS S:\Dhanush\cryptography and network security\Cycle 3> python MD5.py
PS S:\Dhanush\cryptography and network security\Cycle 3> python SHA512.py
Enter Text: I'm travelling 80km per day so if i calculate for a month means it is 2400km.
SHA512 Hash Value:
850105455c70d769a730edf196dcaecc76f1a5e72bcc6bc432fe90a15a68f8d9b5d7fe2eae3fa5fb5eafd493e2077a66f611c4
fe864c2c4425f7e20b75917534
PS S:\Dhanush\cryptography and network security\Cycle 3> python SHA1.py
Enter Text: I need a bike to travel this distance daily
SHA1 Hash Value:
b9cfa284e2ff1a02a8b76d0f841a0025a30f3cca
PS S:\Dhanush\cryptography and network security\Cycle 3>
```

The status bar at the bottom indicates the file is encoded in UTF-8, uses CRLF line endings, and is a Python 3.13 (64-bit) file. The system tray shows the time as 7:14 PM on 8/18/2026.

RESULT

Thus, the SHA1 hash function was implemented successfully and the hash value was generated.

4. DIGITAL SIGNATURE

AIM

To implement a Digital Signature algorithm for signing and verifying a message using Python.

ALGORITHM

1. Start the program.
2. Read the message from the user.
3. Generate the hash value of the message.
4. Create a digital signature using the private key.
5. Verify the signature using the public key.
6. Display the signature.
7. Display the verification result.
8. Stop the program.

PYTHON CODE

```
import hashlib

message = input("Enter Message: ")

n = 143
e = 7
d = 103

hash_value = int(hashlib.sha1(message.encode()).hexdigest(), 16)

signature = pow(hash_value, d, n)

print("Digital Signature:", signature)
```

```
verified_hash = pow(signature, e, n)
```

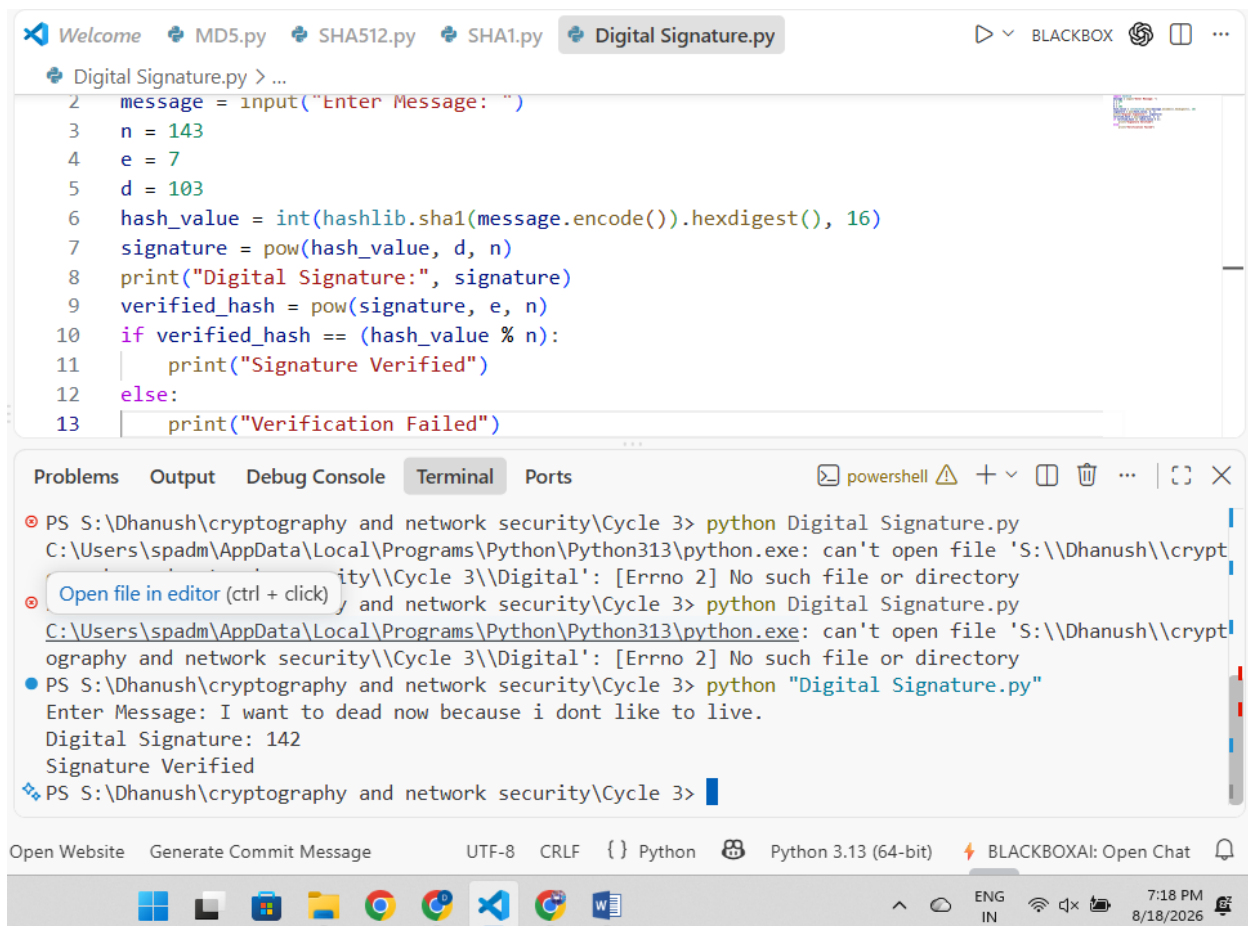
```
if verified_hash == (hash_value % n):
```

```
    print("Signature Verified")
```

```
else:
```

```
    print("Verification Failed")
```

SAMPLE OUTPUT



The screenshot displays a code editor with a file named `Digital Signature.py` open. The code implements a digital signature verification process using SHA-1 hashing and modular exponentiation. The terminal output shows the execution of the script, where the user enters a message, the program calculates a digital signature (142), and successfully verifies it.

```
Welcome MD5.py SHA512.py SHA1.py Digital Signature.py
Digital Signature.py > ...
2 message = input("Enter Message: ")
3 n = 143
4 e = 7
5 d = 103
6 hash_value = int(hashlib.sha1(message.encode()).hexdigest(), 16)
7 signature = pow(hash_value, d, n)
8 print("Digital Signature:", signature)
9 verified_hash = pow(signature, e, n)
10 if verified_hash == (hash_value % n):
11     print("Signature Verified")
12 else:
13     print("Verification Failed")
```

Terminal Output:

```
PS S:\Dhanush\cryptography and network security\Cycle 3> python Digital Signature.py
C:\Users\spadm\AppData\Local\Programs\Python\Python313\python.exe: can't open file 'S:\\Dhanush\\crypt
ity\\Cycle 3\\Digital': [Errno 2] No such file or directory
PS S:\Dhanush\cryptography and network security\Cycle 3> python Digital Signature.py
C:\Users\spadm\AppData\Local\Programs\Python\Python313\python.exe: can't open file 'S:\\Dhanush\\crypt
ography and network security\\Cycle 3\\Digital': [Errno 2] No such file or directory
PS S:\Dhanush\cryptography and network security\Cycle 3> python "Digital Signature.py"
Enter Message: I want to dead now because i dont like to live.
Digital Signature: 142
Signature Verified
PS S:\Dhanush\cryptography and network security\Cycle 3>
```

RESULT

Thus, the Digital Signature generation and verification process was implemented successfully using Python.